

Custom Method Resolution

Implement a class `Child` that inherits from two already implemented classes, `Parent1` and `Parent2`. Both parent classes implement two methods: `fun1` and `fun2`.

In the `Child` class:

- `fun1` should call `Parent1's fun1` before `Parent2's fun1`
- `fun2` should call `Parent2's fun2` before `Parent1's fun2`

Example

```
class Parent1:
    def fun1(self):
        print("Parent1's fun1() method")

    def fun2(self):
        print("Parent1's fun2() method")

class Parent2:
    def fun1(self):
        print("Parent2's fun1() method")

    def fun2(self):
        print("Parent2's fun2() method")
```

Let `c` be the object of class `Child`. `c.fun1()` should print the following:

```
Parent1's fun1() method
Parent2's fun1() method
```

`c.fun2()` should print the following:

```
Parent2's fun2() method
Parent1's fun2() method
```

Function Description

Complete the class `Child` in the editor below with the methods as described.

Constraints

- $1 \leq n \leq 10$
- $1 \leq \text{arr}[i] \leq 2$

Input Format For Custom Testing

The first line contains an integer, n , denoting the number of elements in `arr`.

Each line i of the n subsequent lines (where $0 \leq i < n$) contains an integer describing `arr[i`].

Sample Case 0

Sample Input For Custom Testing

STDIN	Function
-----	-----
2	size of arr[] n = 2
1	arr = [1, 2]
2	

Sample Output

Child fun1
Parent1 fun1
Parent2 fun1
Child fun2
Parent2 fun2
Parent1 fun2

Explanation

The first call is `c.fun1()`, and the second call is `c.fun2()`, where `c` is an object of class `Child`.

Sample Case 1

Sample Input For Custom Testing

STDIN	Function
-----	-----
2	size of arr[] n = 2
2	arr = [2, 2]
2	

Sample Output

Child fun2
Parent2 fun2
Parent1 fun2
Child fun2
Parent2 fun2
Parent1 fun2

Explanation

The first call is `c.fun2()`, and the second call is `c.fun2()`, where `c` is an object of class `Child`.